

Contents

SkillOS : Claude Code Skills を汎用 AI エンジニアリング実行基盤へ固化するためのアーキテクチャ分析	2
1. エグゼクティブサマリー	2
2. 現在の Skills をシステム能力として再定義する	3
3. Claude Code をそのまま Web に移植してはいけない理由	5
4. 推奨する全体アーキテクチャ	5
5. Skill はアプリケーションへハードコードしない	7
6. Skill Registry に持たせるべきメタデータ	8
7. Playbook を Markdown から Executable Object へ進化させる	8
8. PDCA は独立した Engine にする	10
9. Web UI は Chat 中心ではなく Task 中心にする	11
10. Task Run 画面を製品の中心にする	12
11. API Key はブラウザから直接モデル API に送らない	13
12. Platform Key と BYOK の両方を用意する	14
12.1 Platform Key モデル	14
12.2 BYOK モデル	14
13. Credential Vault を独立機能として設計する	14
14. Permission Engine は必須	15
Auto Allow	15
Ask User	15
Never Allow	16
15. Hook は Skill と分離する	16
16. Skill Router が汎用化の中心になる	17
17. Multi-Agent を前提にデータモデルを設計する	18
18. 推奨データモデル	18
19. Workspace を Task の実体として扱う	20
20. Runtime は二層構造を推奨する	21
21. Managed Runtime に向く処理	21
22. Self-hosted Runtime が必要になるケース	21
23. Model Provider は抽象化する	22
24. Skill Registry を Source of Truth にする	22
25. Multi-Tenant を最初から考慮する	23
26. Domain Skill を Capability Pack に進化させる	24
27. Capability Marketplace への発展	24

28. 本当に蓄積すべき知的資産	25
29. 実行閉ループを完全な形にする	25
30. 第二の PDCA : Skill PDCA	26
31. V1 の範囲は意図的に狭くする	27
32. V1 に必要な主要画面	27
33. V1 推奨技術スタック	28
34. Acceptance Contract を中心に据える	29
35. Evidence Bundle を標準成果物にする	30
36. 製品コンセプト : SkillOS	30
37. Claude Code、Web、API の関係	31
38. 推奨するシステム境界	32
39. 実装優先順位	33
P0 : Core Runtime	33
P1 : Production Foundation	33
P2 : Intelligent Orchestration	33
P3 : Platform Ecosystem	33
40. 最終評価	34
41. 推奨する V1 開発 Playbook	34
付録 A : V1 の最小実行フロー	35
付録 B : 設計原則	35
付録 C : 最終到達イメージ	36

SkillOS : Claude Code Skills を汎用 AI エンジニアリング実行基盤へ固化するためのアーキテクチャ分析

版: 1.0

作成日: 2026-10-02

対象: Claude Code 上で蓄積した Skills、Playbook、PDCA、ドメイン知識を Web/API ベースの汎用アプリケーションへ移行する構想

1. エグゼクティブサマリー

Claude Code 上で、以下のような能力がすでに蓄積されている場合、単なる「AI コーディング支援」を超えて、汎用的な **AI Engineering Execution Platform** を構築できる段階に入っている。

- Playbook 作成 Skill
- Playbook 実行 Skill
- タスク状態評価 Skill
- PDCA Skill
- ソフトウェア開発 Skills
- テスト・品質保証 Skills

- デプロイ・運用 Skills
- Git / Docker / Cloud / MCP などの Tool Skills
- 製造、エネルギー、研究、医療、自動車、金融などの Domain Skills

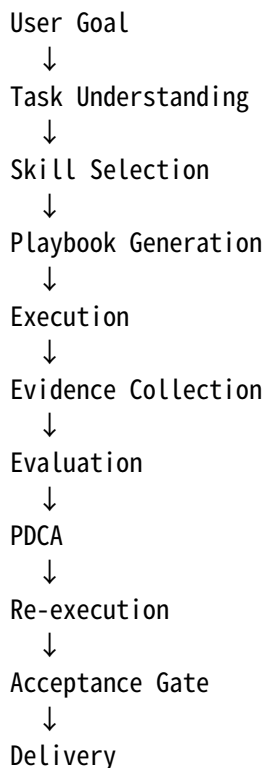
ただし、重要なのは **Claude Code 自体を Web 化することではない**。

正しい方向は、Claude Code の中に暗黙的に存在している以下の要素を独立した実行基盤として切り出すことである。

Agent Runtime
Skill Runtime
Playbook Runtime
Workspace Runtime
Tool Runtime
PDCA Runtime
Security / Governance Runtime

その上で Web、API、VS Code、Slack、LINE、モバイルなどをクライアントとして接続する。

最終的な目標は次の処理を一貫して実行できることにある。



この構想の本質は、「Claude を Web から呼び出せるようにする」ことではなく、**知識・手順・評価・実行証跡をモデルから分離し、再利用可能な実行資産として管理することにある**。

2. 現在の Skills をシステム能力として再定義する

現在の Skill 群は、製品化する際には次のように抽象化するとよい。

既存の能力	製品化後の役割
Playbook Creation Skill	Planner / Architect
Playbook Execution Skill	Executor
Task Status Evaluation Skill	Evaluator
PDCA Skill	Controller / Optimizer
Software Development Skills	Engineering Capability
Testing Skills	Verification Capability
Deployment Skills	Release Capability
Domain Skills	Domain Capability Packs
Git / Docker / Cloud / MCP Skills	Tool Capability
文書・レポート生成 Skills	Artifact Capability

従来の発想は、概ね以下のようなものである。

Claude



コードを書く

これを次のような実行モデルへ変える。

User Goal



Task Interpreter



Skill Router



Domain Skills



Playbook Generator



Playbook Executor



Evidence Collector



Evaluator



PDCA Controller



Pass?

No

Yes



Adjust Plan

Deliver



Execute again

この形になると、システムは単なる AI アシスタントではなく、**Autonomous Work Engine** または **AI Engineering Execution Engine** として扱える。

3. Claude Code をそのまま Web に移植してはいけない理由

Claude Code の価値は UI ではなく、以下の組み合わせにある。

LLM Reasoning
+
Filesystem
+
Shell / Code Execution
+
Tools / MCP
+
Agent Loop

Skill は、モデルに対して以下を与える役割を持つ。

- どの状況で動くか
- どの手順を使うか
- どのツールを使うか
- どの制約を守るか
- 何をもって完了とするか
- 失敗時にどう修正するか

したがって、製品化の際に抽出すべきなのは Claude Code の画面ではなく、内部の実行原理である。

Claude Code

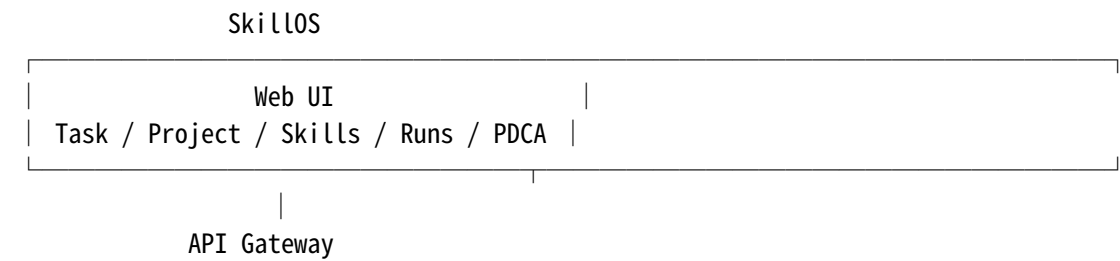


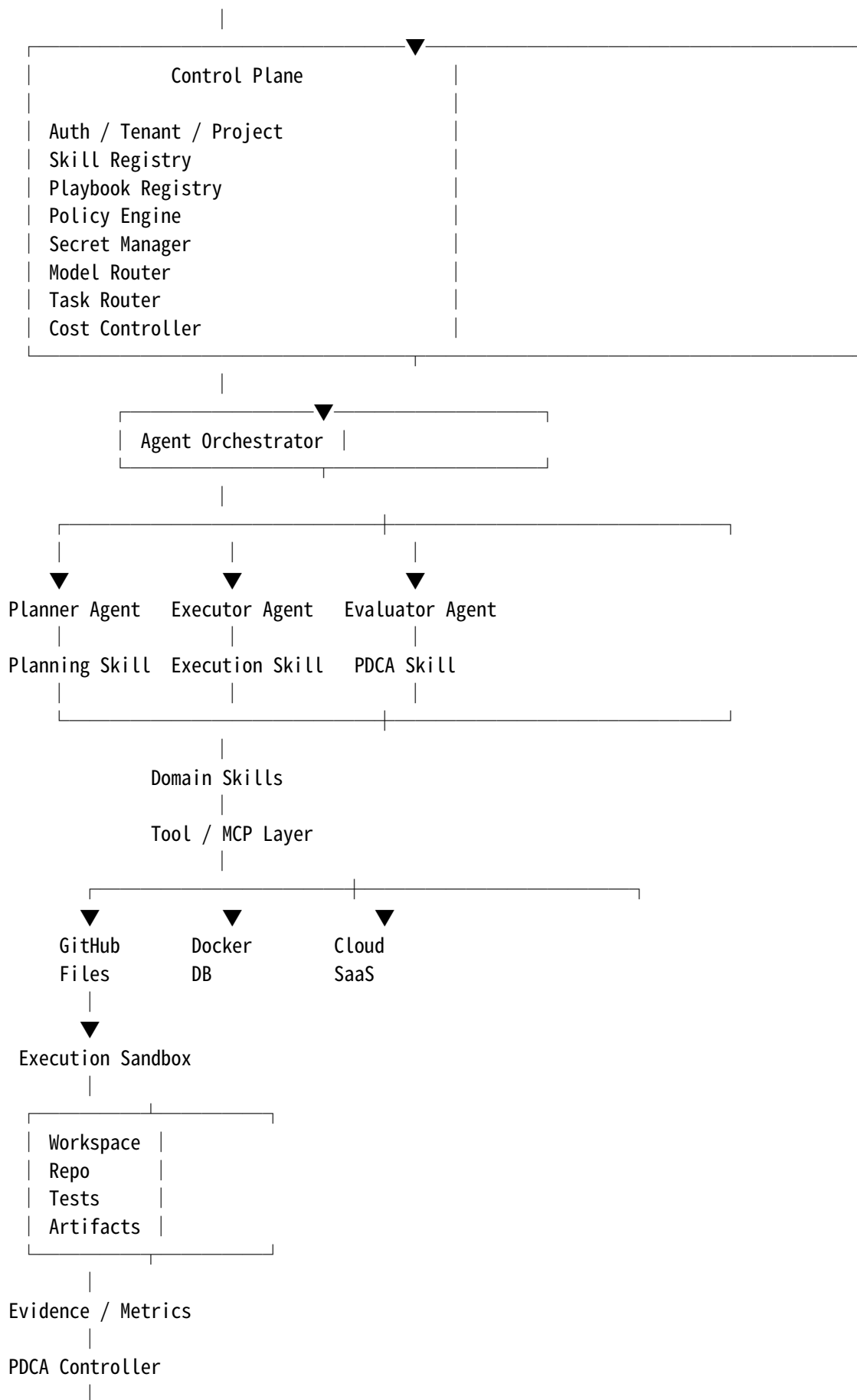
Agent Runtime	
Skill Runtime	
Tool Runtime	
Workspace Runtime	
Playbook Runtime	
PDCA Runtime	
Security Runtime	

Web はこの Runtime に接続する一つのクライアントにすぎない。

4. 推奨する全体アーキテクチャ

推奨は、独自の **Control Plane** と、Claude などのモデル・Agent Runtime を接続する **Runtime Adapter** を分離する構造である。





└── FAIL → Re-plan
└── PASS → Deliver

この構造の利点は、モデル、実行環境、UI、業務領域を疎結合にできることである。

5. Skill はアプリケーションへハードコードしない

避けるべき設計は次のようなものである。

```
if task == "create_playbook":  
    prompt = "..."
```

Skill をコード内の条件分岐や巨大 Prompt に埋め込むと、更新、再利用、監査、バージョン管理が困難になる。

推奨構造は次の通りである。

Skill Registry
↓
Skill Resolver
↓
Skill Loader
↓
Agent Runtime

Skill の論理構造例：

```
skills/  
├── core/  
│   ├── playbook-create/  
│   ├── playbook-execute/  
│   ├── pdca/  
│   ├── task-evaluate/  
│   └── artifact-package/  
├── software/  
│   ├── architecture/  
│   ├── frontend/  
│   ├── backend/  
│   ├── database/  
│   ├── test/  
│   ├── security/  
│   └── deployment/  
├── cloud/  
│   ├── aws/  
│   ├── azure/  
│   └── gcp/  
└── domain/  
    ├── energy/  
    ├── manufacturing/  
    └── research/
```

```
|—— medical/  
|—— automotive/  
|—— finance/
```

Skill は Prompt ではなく、再利用可能な **実行資産** として扱うべきである。

6. Skill Registry に持たせるべきメタデータ

各 Skill には最低限、以下のメタデータを持たせる。

```
skill_id  
name  
version  
description  
category  
domain  
capabilities  
triggers  
dependencies  
required_tools  
required_credentials  
risk_level  
input_schema  
output_schema  
evaluation  
owner  
status
```

これにより以下が可能になる。

- バージョン管理
- Release / Rollback
- レビュー
- Benchmark
- A/B テスト
- Dependency 解決
- Runtime ごとの配布
- 使用実績分析
- Skill の品質評価

Skill Registry を単なるファイル一覧ではなく、**Capability Registry** として設計することが重要である。

7. Playbook を Markdown から Executable Object へ進化させる

従来の Playbook は人間が読む文書として十分でも、Runtime が安全かつ再現可能に実行するには構造化が必要である。

例：

PB-001

ログイン機能を改善する。

手順：

1. 現在のコードを確認
2. API を修正
3. フロントエンドを修正
4. テストを追加
5. テストを実行

これを内部では、以下のような DSL に変換する。

playbook_id: PB-001

goal:

implement_login

requirements:

- JWT
- RBAC
- refresh_token

steps:

- id: S01
action: analyze
skills:
 - backend-analysis
 - security
- id: S02
action: implement
skills:
 - backend
 - authentication
- id: S03
action: test
skills:
 - automated-test
- id: S04
action: evaluate
skills:
 - pdca

acceptance:

tests_pass: true
coverage: ">=80%"
security_critical: 0

artifacts:

- source_code
- test_report
- evidence_bundle

人間向けには Markdown 表示を維持し、実行時には DSL を使用する。

これにより以下が可能になる。

- Pause
 - Resume
 - Retry
 - Partial Execution
 - Delta Execution
 - Rollback
 - Replay
 - Audit
 - Deterministic Validation
-

8. PDCA は独立した Engine にする

PDCA を単純な「Claude にもう一度レビューさせる処理」にすると、品質もコスト効率も低下する。

推奨構造は次の通りである。

PLAN

↓

Goal / Playbook / Acceptance Criteria

DO

↓

Execute

CHECK

↓

Tests

Lint

Security

Build

Runtime

Requirements

Artifacts

Benchmark

ACT

↓

Gap Analysis

↓

Generate Delta Playbook

↓

Execute again

特に重要なのは、PDCA の再実行単位を **全 Playbook** ではなく **差分 Playbook** にすることである。

例：

PB001～PB040

Initial Run

Pass: 36

Fail: 4

PDCA が生成するのは次の修正のみでよい。

PB041-FIX

PB042-FIX

PB043-FIX

PB044-FIX

次の実行で 1 件だけ失敗した場合：

Pass: 39

Fail: 1

さらに：

PB045-FIX

を生成する。

最終的に：

Acceptance Gate = PASS

となった時点で完了とする。

この構造により、Token、実行時間、再修正範囲、回帰リスクを大きく削減できる。

9. Web UI は Chat 中心ではなく Task 中心にする

一般的な Chat UI は「会話」には向いているが、「長時間の実行管理」には向いていない。

推奨するトップレベル UX は **New Task** である。

例：

項目	例
Task	システムを Production Ready にする
Project	対象プロジェクト
Repository	GitHub
Input	ZIP / Git
Domain	Software / Manufacturing / Research など
Objective	Production Ready
Model	Claude
Skills	Auto
Execution	Automatic
PDCA	Enabled
Max Iteration	5

項目	例
Approval	Important Actions
Budget	上限設定

実行後は、以下のステータスを表示する。

TASK-20261002-001

Understanding ✓
Skill Selection ✓
Architecture ✓
Playbook ✓
Implementation ●
Testing
Evaluation
PDCA
Release

10. Task Run 画面を製品の中心にする

Task Run 画面では、少なくとも以下を確認できるようにする。

UI	内容
Task	現在のゴール
Plan	実行計画
Skills	実際に選択された Skill
Playbook	各 PB の状態
Files	変更されたファイル
Diff	Git 差分
Terminal	実行コマンド
Tests	テスト結果
Evidence	実行証跡
PDCA	現在の Cycle
Cost	Token / API / Compute
Artifacts	最終成果物

表示例：

TASK
Platform Upgrade

Progress
 78%

Playbooks
PB001 ✓

PB002 ✓
PB003 ✓
PB004 NG
PB005 ✓

PDCA
Cycle 2 / 5

Current issue
PB004 API integration test failed

Agent action
Investigating API contract mismatch...

Files changed
backend/api/order.py
tests/api/test_order.py

Commands
pytest tests/api/test_order.py

Result
17 passed
1 failed

この構造は、CLI を直接見せるよりも、非開発者、PM、経営層、顧客にとって理解しやすい。

11. API Key はブラウザから直接モデル API に送らない

避けるべき構造：

Browser
↓
Model API

また、API Key を localStorage に保存する設計も避ける。

推奨：

Browser
|
▼
Application Backend
|
|—— Auth
|—— Tenant
|—— Credential Vault
▼
Model Provider API

ユーザーが入力した Key は、サーバー側で暗号化する。

API Key



Encrypt



KMS



Secrets Manager / Encrypted Database

画面にはマスク済み情報のみを表示する。

Provider

Connected

Key

.....4eF

12. Platform Key と BYOK の両方を用意する

12.1 Platform Key モデル

プラットフォーム側の API Key を使用する。

User



Platform



Model Provider

ユーザーには実行回数、Token、プラン、月額利用量などで課金する。

12.2 BYOK モデル

Bring Your Own Key

企業顧客自身が API Key を登録する。

Customer



Platform



Customer Credential



Model Provider

企業導入では、BYOK の選択肢が重要になる可能性が高い。

13. Credential Vault を独立機能として設計する

将来必要になる Secret はモデル API Key だけではない。

MODEL_API_KEY
AWS_ACCESS_KEY
AWS_SECRET_KEY
GITHUB_TOKEN
DATABASE_URL
DOCKER_TOKEN
SLACK_TOKEN
LINE_TOKEN
MCP Credentials

したがって、「API Key 設定画面」ではなく **Credential Vault** として設計する。

データモデル例：

Credential

id
tenant_id
provider
credential_type
encrypted_value
scope
allowed_projects
created_at
last_used_at
expires_at

14. Permission Engine は必須

汎用 Agent の最大リスクは、判断能力不足よりも **過剰権限** である。

推奨する権限区分：

Auto Allow

Read File
Search
Run Tests
Lint
Build
Install approved package

Ask User

Write production database
Git Push
Deploy production
Delete cloud resource
External upload
Change access control

Never Allow

Read unrelated secrets
Delete host filesystem
Bypass tenant isolation
Unapproved data exfiltration
Disable audit logs

Permission Engine がなければ、システムは「便利な AI」ではなく、高権限の Remote Shell に近づいてしまう。

15. Hook は Skill と分離する

Skill はモデルが判断して使う **知的ルール** である。

一方 Hook は、必ず実行される **決定的ルール** として扱う。

例：

SessionStart
PreToolUse
PostToolUse
PermissionRequest
Stop

実際の制御：

PreToolUse
↓
Security Check

PostToolUse
↓
Evidence Collection

File Modified
↓
Lint / Format

Test Completed
↓
PDCA Evaluation

Task Stop
↓
Acceptance Gate

設計原則は次の通りである。

Skill = Intelligent Rule
Hook = Deterministic Rule

16. Skill Router が汎用化の中心になる

Skill 数が数百、数千に増えた場合、すべてを毎回読み込むことはできない。

推奨：

```
Task
↓
Task Classifier
↓
Skill Router
↓
Candidate Skills
↓
Semantic Ranking
↓
Select Skills
↓
Runtime
```

例えば：

既存システムを、製造業向けの本番運用可能な管理システムへアップグレードする。

というタスクなら、内部では次のように分類する。

```
{
  "task_type": "software_upgrade",
  "domains": [
    "software",
    "manufacturing"
  ],
  "capabilities": [
    "architecture",
    "backend",
    "frontend",
    "database",
    "testing",
    "deployment"
  ]
}
```

その結果、必要な Skill のみをロードする。

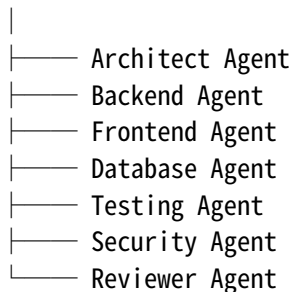
```
core.playbook-create
core.playbook-execute
core.pdca
software.architecture
software.backend
software.frontend
software.database
software.testing
domain.manufacturing
```

これにより、基盤モデルを **動的に構成される専門 Agent** として利用できる。

17. Multi-Agent を前提にデータモデルを設計する

複雑なタスクは、一つの Agent に全責任を持たせるより、役割分担した方が管理しやすい。

Coordinator



各 Agent が異なるものを持てるようにする。

Skills

Tools

Context

Permission

Model

Budget

最終的には：

Coordinator

↓

Integration

↓

Evaluation

↓

PDCA

とする。

データモデルは最初から：

1 Task = 1..N Agents

をサポートする方がよい。

18. 推奨データモデル

PostgreSQL を中心にする場合、最低限以下のテーブル群を持つ。

organizations

users

projects

providers

credentials

repositories
workspaces

skills
skill_versions
skill_dependencies

agents
agent_skills

tasks
task_runs

playbooks
playbook_versions
playbook_steps
step_runs

pdca_cycles

tool_calls

files
artifacts

evaluations
evidence

approvals

audit_logs

usage
costs

特に重要な追跡チェーン：

```
Task
├─ Run
│   └─ Playbook
│       └─ Step
│           └─ ToolCall
│               └─ Evidence
```

これにより、例えば次の問いに答えられる。

なぜこの機能は完了と判定されたのか。

回答は：

PB203
↓

Step S07
↓
pytest
↓
127 passed
↓
artifact SHA256
↓
commit abc123
という形で追跡できる。

19. Workspace を Task の実体として扱う

各 Task は Chat History だけでなく、独立した作業領域を持つべきである。

```
/workspace  
  /repo  
  /input  
  /output  
  /artifacts  
  /logs  
  /evidence  
  /playbooks  
  /reports
```

例：

```
tasks/TASK-001/
```

```
  task.yaml
```

```
  playbooks/  
    PB001.md  
    PB002.md
```

```
  repo/
```

```
  evidence/  
    build.json  
    tests.json
```

```
  artifacts/  
    result.zip
```

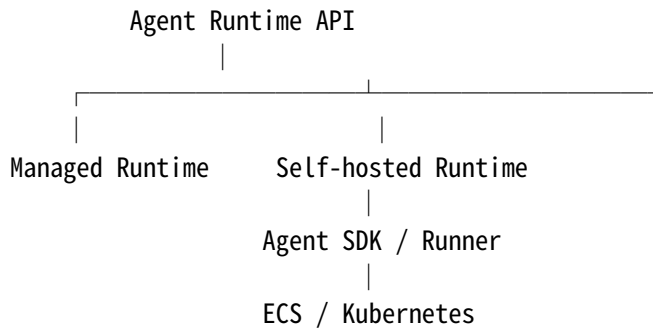
```
  pdca/  
    cycle-001.md  
    cycle-002.md
```

Workspace は、Claude Code のローカル作業ディレクトリをサーバー側で再構成したものに近い。

20. Runtime は二層構造を推奨する

第一段階では Managed Runtime を利用し、開発量を抑える。

第二段階では Self-hosted Runtime を追加する。



Runtime Profile も抽象化しておく。

FAST
STANDARD
ENTERPRISE
GPU
PRIVATE

21. Managed Runtime に向く処理

Managed Runtime は以下に向く。

一般的な Web 開発
Git Repository 分析
通常の Build / Test
文書生成
Playbook 実行
一般的な PDCA

これにより V1 のインフラ実装負担を減らせる。

22. Self-hosted Runtime が必要になるケース

次の要件がある場合は自前 Runtime が必要になる。

企業内ネットワーク
Private Git
大型 DB
GPU
特殊 SDK
特殊コンパイラ

大規模シミュレーション
専門的な科学計算ツール
厳格なデータ所在地要件

候補：

Agent SDK

+

Docker

+

ECS Fargate

+

Kubernetes

+

必要に応じて MicroVM / Sandbox

23. Model Provider は抽象化する

V1 は Claude のみでも問題ない。

しかし内部モデルは：

ModelProvider

とし、特定ベンダー名に固定しない方がよい。

例：

Planner → Claude

Coder → Claude

Reviewer → Claude

Vision → Another Model

Local Task → Local Model

長期的に価値を持つ資産は：

Skill

Playbook

Evaluation

Evidence

であり、モデル自体ではない。

24. Skill Registry を Source of Truth にする

各 Skill はプラットフォーム固有 ID を持つ。

skill_id:

skill-playbook-pdca

version:

3.2.1

各 Runtime へのマッピングは別管理する。

provider_mapping:

anthropic:

skill_id: provider_skill_xxx

version: 7

local:

package: skill-playbook-pdca-3.2.1

これにより、外部 Runtime は配布先でしかなくなる。

Skill Registry 自体を Source of Truth にする。

25. Multi-Tenant を最初から考慮する

避けるべき構造：

All Customers

↓

One Workspace

↓

One Skill Namespace

推奨：

Tenant

|

├ Skill Namespace

├ Credential Vault

├ Projects

├ Workspaces

├ Artifacts

└ Runtime Namespace

分離対象：

- Skill
 - Secret
 - Project
 - Workspace
 - Artifact
 - Audit Log
 - Cost
 - Runtime
 - Model Usage
-

26. Domain Skill を Capability Pack に進化させる

個別 Skill だけではなく、ある領域の実行能力一式を **Capability Pack** としてまとめる。

例：

Manufacturing Pack

内部：

Domain Knowledge

Skills

Playbook Templates

Evaluation Rules

Tools

MCP

Schemas

Examples

Benchmark

他にも：

Energy Pack

Scientific Research Pack

Healthcare Pack

Automotive Pack

Enterprise Modernization Pack

Data Center Pack

などが考えられる。

利用者から見ると：

Install Capability Pack

だけで、汎用 Agent が特定業務向け Agent に変化する。

General Agent

+

Manufacturing Pack

=

Manufacturing Engineering Agent

27. Capability Marketplace への発展

Capability Pack が増えた場合、Marketplace に発展させられる。

Core

Software Engineering

Cloud

Docker

PostgreSQL

Industry

Manufacturing
Energy
Healthcare
Automotive
Scientific Research

Compliance
ISO 27001
ISO 26262
ASPICE
Privacy
Safety

組織ごとに必要な Pack を追加できるようになる。

28. 本当に蓄積すべき知的資産

長期的な競争力は基盤モデルそのものではない。

蓄積すべきものは：

Skill Library
Playbook Library
Evaluation Library
Evidence Library

モデルは更新されるが、以下の知識は組織資産として残る。

どの順番で設計するか
どこを検証するか
どの条件なら再実行するか
どうすれば Production Ready と判断できるか
どういう失敗が多いか
どの修正が有効か

これらを Skill と Playbook に固定することが、プラットフォーム化の最大の価値である。

29. 実行閉ループを完全な形にする

基本構造：

Skill
↓
Playbook
↓
Execution
↓
PDCA

推奨する完成形：

GOAL
↓
Capability Discovery
↓
Skill Composition
↓
PLAN
↓
Executable Playbook
↓
EXECUTION
↓
Evidence
↓
Evaluation
↓
PDCA
↓
Learning
↓
Skill / Playbook Improvement

「実行して終わり」ではなく、実行結果を次の Skill 改善に還元する。

30. 第二の PDCA : Skill PDCA

第一層：

Task PDCA

現在のタスクを改善する。

第二層：

Skill PDCA

プラットフォームそのものを改善する。

例：

1000 Runs
↓
Failure Clustering
↓
Common Causes
↓
Skill Weakness
↓
Generate Improvement
↓
Benchmark
↓

A/B Test

↓

Promote

例：

Skill v2 success = 62%

Skill v3 success = 94%

Benchmark が十分であれば、v3 を Production へ昇格させる。

これにより、Skill 基盤自体が実行回数とともに成熟する。

31. V1 の範囲は意図的に狭くする

最初からあらゆる業務を対象にすると、Runtime、権限、UI、評価が複雑化する。

V1 は一つの非常に明確なユースケースに絞るべきである。

ユーザー：

ZIP をアップロード

または

Git Repository を接続

入力：

このシステムを分析し、Production Ready まで改善してください。

システム：

1. Analyze
2. Select Skills
3. Create Playbooks
4. Execute
5. Build / Test
6. Evaluate
7. PDCA
8. Re-execute
9. Generate Report
10. Download ZIP / Create PR

これだけでも十分に独立した製品価値がある。

32. V1 に必要な主要画面

画面	機能
Dashboard	Project / Task / Run 一覧
New Task	新規タスク作成
Task Run	実行状況、Playbook、PDCA
Project	Repo / Files / Artifacts

画面	機能
Skills	Skill Library
Credentials	Credential Vault
Settings	Model / Runtime / Policy

最重要画面は **Task Run** である。

33. V1 推奨技術スタック

一例：

Frontend
Next.js

Backend
FastAPI

Database
PostgreSQL

Queue
Redis

Artifacts
S3-compatible Object Storage

Auth
Cognito / Keycloak / OIDC

Secrets
KMS
+
Secrets Manager

Runtime V1
Managed Agent Runtime

Runtime V2
Agent SDK
+
ECS Fargate

Large / Private Runtime
Kubernetes

Source Integration
GitHub / GitLab

Observability
OpenTelemetry
+
LLM Trace Platform

Deployment
Container Platform
+
Load Balancer
+
Managed Database
+
Object Storage

34. Acceptance Contract を中心に据える

タスク完了を「モデルが Done と答えた」ことで判断してはいけない。

各 Task に対して構造化された Acceptance Contract を作る。

`acceptance_contract:`

```
functional:  
  - requirement_coverage >= 0.95
```

```
build:  
  - success
```

```
tests:  
  - pass_rate = 1.0
```

```
quality:  
  - critical_issue = 0
```

```
security:  
  - critical = 0
```

```
artifacts:  
  - documentation  
  - source  
  - test_report
```

```
runtime:  
  - health_check = pass
```

Evaluator と PDCA は、この Contract を基準に動く。

これによってプラットフォームは：

AI Coding Tool

から：

AI Engineering Execution Platform

へ進化する。

35. Evidence Bundle を標準成果物にする

実行完了時に返すものは Done だけでは不十分である。

標準成果物例：

Result

```
|—— source.zip
|—— patch.diff
|—— build-report.json
|—— test-report.html
|—— security-report.json
|—— requirement-traceability.md
|—— pdca-report.md
|—— execution-log.json
|—— playbooks/
|—— evidence/
```

Evidence Bundle により：

- 再現性
- 監査性
- 説明可能性
- 顧客レビュー
- 品質保証
- 規制対応

を強化できる。

36. 製品コンセプト：SkillOS

この基盤は、特定モデル専用の「開発プラットフォーム」ではなく、より一般的な **AI Execution Platform** として位置付ける方がよい。

概念モデル：

Model

= Intelligence

Skills

= Expertise

Playbooks

= Procedures

Tools
= Actions

Workspace
= World

Evidence
= Facts

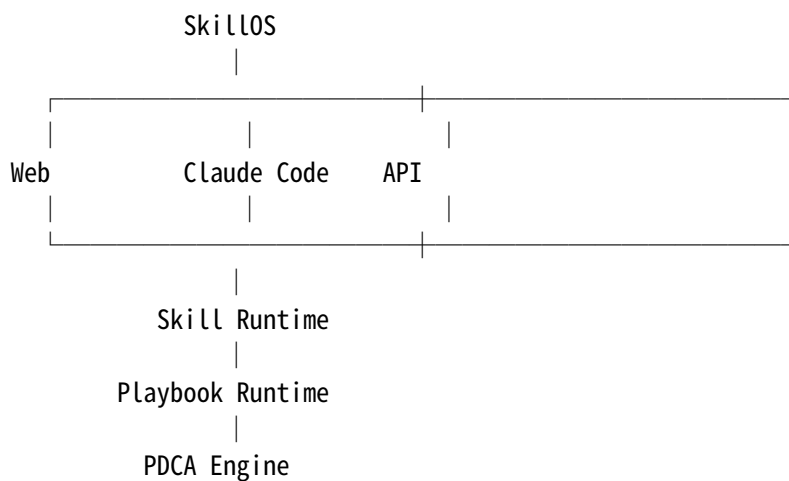
PDCA
= Control

SkillOS
= Operating System

この抽象化により、モデルや UI が変わっても中核資産を維持できる。

37. Claude Code、Web、API の関係

将来的には次の構造が望ましい。



つまり Claude Code はプラットフォームそのものではなく、SkillOS を利用する一つのクライアントになる。

将来的には：

VS Code
Slack
LINE
Mobile App
External API

なども同じ Runtime を使用できる。

38. 推奨するシステム境界

製品の中核は以下の 10 層で整理できる。

Layer	役割
L1 Model	Reasoning / Generation
L2 Skill	専門知識・実行ルール
L3 Playbook	タスクの実行計画
L4 Orchestrator	Agent / Step 編成
L5 Runtime	ファイル・コード・ツール実行
L6 Evidence	実行証拠
L7 Evaluator	状態評価
L8 PDCA	自動修正
L9 Governance	権限・安全・監査
L10 UI / API	ユーザー入口

任意の業務 Agent は次の式で構成できる。

General Platform
+
Domain Capability Pack
+
Domain Tools
+
Domain Evaluation

例：

General Platform
+
Manufacturing Pack
=
Manufacturing Agent

General Platform
+
Scientific Research Pack
=
Research Agent

General Platform
+
Energy Pack
=
Energy Engineering Agent

この構造なら、個別製品を無限に作るのではなく、**共通 Execution Kernel + Capability Pack** へ収束できる。

39. 実装優先順位

P0 : Core Runtime

Skill Registry
+
Playbook Runtime
+
Task Runtime
+
PDCA Runtime
+
Web UI
+
Model / Runtime Adapter

P1 : Production Foundation

Workspace
+
Evidence
+
Permission Engine
+
Credential Vault
+
Git Integration

P2 : Intelligent Orchestration

Multi-Agent
+
Skill Router
+
Capability Packs
+
Evaluation Framework

P3 : Platform Ecosystem

Skill Marketplace
+
Skill Benchmark
+
Skill Auto-Improvement
+
Multi-model Runtime

40. 最終評価

現時点で最も優先すべきことは、Skill をさらに大量に追加することではない。

最重要なのは：

Skill Runtime を固化すること

である。

戦略としては：

「Claude Code を Web 化する」

のではなく：

Claude Code 上で蓄積した Skill + Playbook + PDCA を独立した SkilloS に抽象化し、Claude Code、Web、API、Managed Runtime、Self-hosted Runtime をすべて実行チャネルとして扱う。

これにより、既存のさまざまなドメイン向け Skill、Playbook、Benchmark、評価ルール、ツール統合を、一つの共通実行基盤へ統合できる。

長期的には、個別システムを作り続けるよりも：

共通 Agent Execution Kernel + Domain Capability Packs

という形へ収束させる方が、再利用性、保守性、商品化可能性、組織知の蓄積という点で有利である。

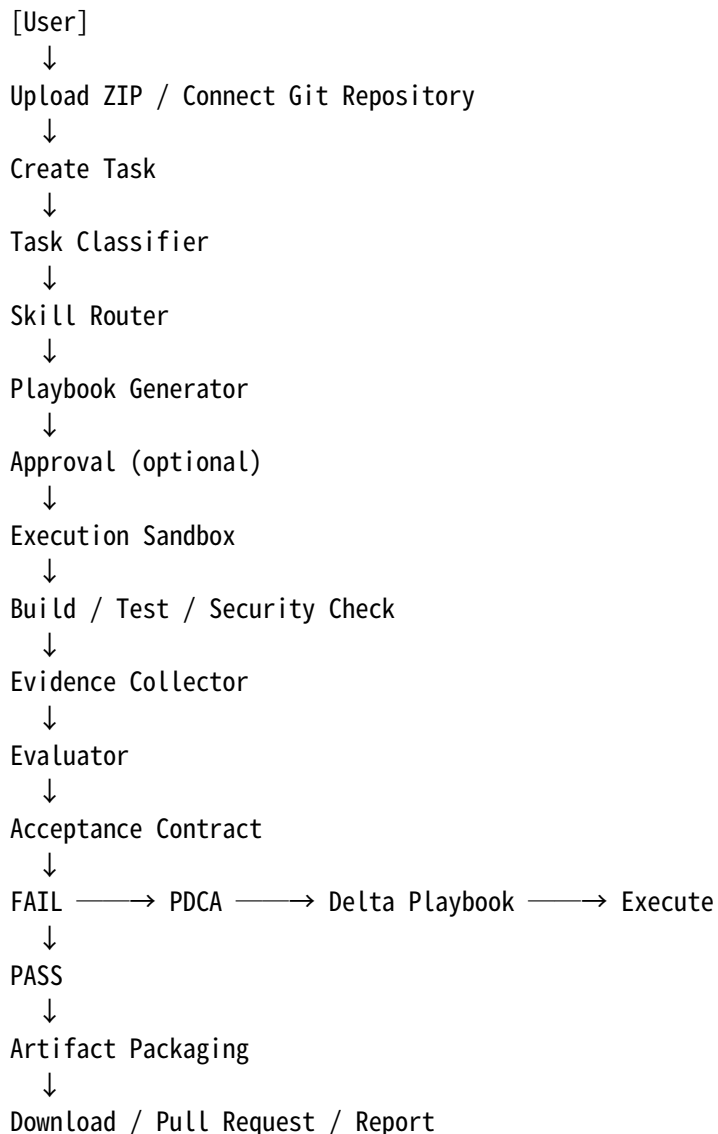
41. 推奨する V1 開発 Playbook

最初の実装単位として、以下の 20 Playbook を推奨する。

SKILLOS-001 Overall Architecture
SKILLOS-002 Skill Registry
SKILLOS-003 Skill Manifest Specification
SKILLOS-004 Task Runtime
SKILLOS-005 Playbook DSL
SKILLOS-006 Playbook Runtime
SKILLOS-007 PDCA Engine
SKILLOS-008 Acceptance Contract
SKILLOS-009 Evidence Bundle
SKILLOS-010 Workspace Runtime
SKILLOS-011 Model Runtime Adapter
SKILLOS-012 Credential Vault
SKILLOS-013 Permission Engine
SKILLOS-014 Web Task Console
SKILLOS-015 Git Integration
SKILLOS-016 Cost & Usage Control
SKILLOS-017 Multi-Tenant Architecture
SKILLOS-018 Audit & Observability
SKILLOS-019 Deployment Architecture
SKILLOS-020 V1 End-to-End Acceptance

この 20 Playbook が完了すると、Claude Code の外でも動作する最初の SkilloS を構築するための骨格が揃う。

付録 A：V1 の最小実行フロー

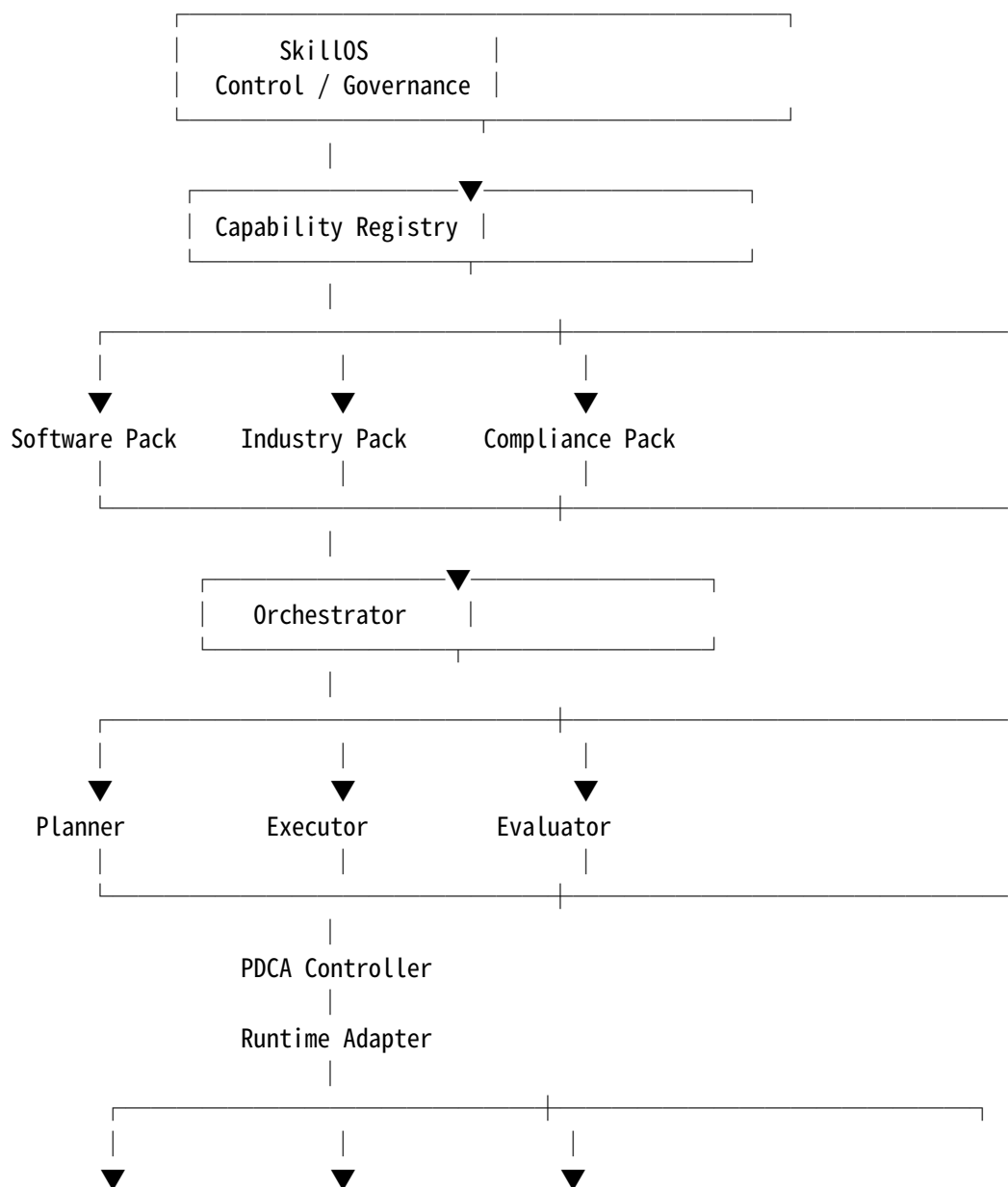


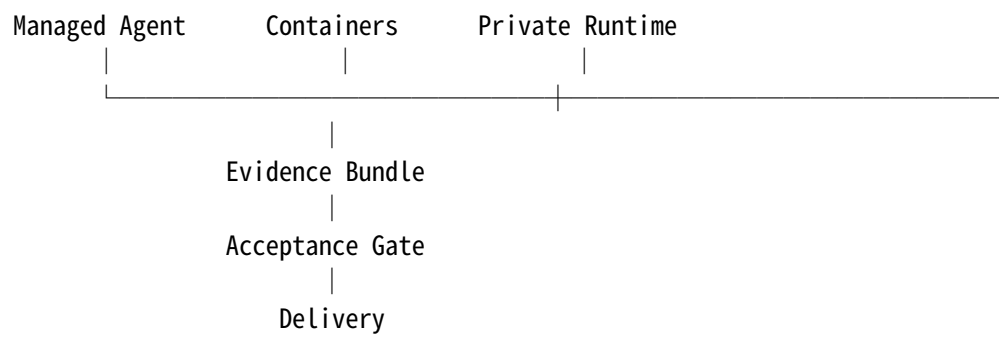
付録 B：設計原則

1. **Model と Know-how を分離する**
モデルは交換可能、Skill と Playbook は組織資産とする。
2. **Chat より Task を中心にする**
長時間実行、状態管理、再開、監査を優先する。
3. **Playbook を実行可能な DSL にする**
Markdown のみで運用しない。
4. **PDCA は差分再実行にする**
全体やり直しを避ける。
5. **Acceptance Contract で完了を定義する**
Agent の自己申告を完了条件にしない。

6. **Evidence** を自動収集する
実行証跡を第一級データとして保存する。
 7. 権限を **Runtime** から分離する
Permission Engine で制御する。
 8. **Credential** を **Vault** 管理する
Browser、Prompt、Log に Secret を残さない。
 9. **Multi-Tenant** を後付けしない
Skill、Workspace、Secret、Artifact を最初から分離する。
 10. **Skill** の品質も **PDCA** 対象にする
実行結果から Skill 自体を継続改善する。
-

付録 C：最終到達イメージ





この形まで到達すれば、Skill は単なる Prompt 集ではなく、**再利用可能・評価可能・配布可能・改善可能な AI 実行コンポーネント**になる。